STAT 184 - Homework 5: Functions, Iteration & Regular Expressions

Due Friday, June 19 at 11:59 PM on Canvas

Your Name Here

AI Disclosure

- ☐ No AI used
- ☐ Syntax / Debugging (e.g., finding a missing comma, fixing an error code)
- ☐ Conceptual explanation (e.g., asking how `{ }` works)
- ☐ Code Generation (e.g., asking AI to write a function from scratch)

Briefly describe your use (1-2 sentences):

[Type your explanation here]

Setup

This week you've been hired as a junior archivist at the **Galactic Records Office**. Your job is to stop copying and pasting the same analysis over and over, and instead build reusable *tools*: functions that you write once, iteration that runs them across many columns and files, and regular expressions that pull order out of messy text.

Run the chunk below to load the tidyverse packages.

```
library(tidyverse)
```

Most of this homework uses `starwars`, a dataset of 87 characters that comes built into `dplyr`. This data is *messy*: it has missing values, numeric columns, character columns full of names, and even **list-columns** (`films`, `vehicles`, `starships`) where a single cell holds a whole vector. Take a look:

```r
glimpse(starwars)
```

```
Rows: 87
Columns: 14
$ name       <chr> "Luke Skywalker", "C-3PO", "R2-D2", "Darth Vader", "Leia Or~
$ height     <int> 172, 167, 96, 202, 150, 178, 165, 97, 183, 182, 188, 180, 2~
$ mass       <dbl> 77.0, 75.0, 32.0, 136.0, 49.0, 120.0, 75.0, 32.0, 84.0, 77.~
$ hair_color <chr> "blond", NA, NA, "none", "brown", "brown, grey", "brown", N~
$ skin_color <chr> "fair", "gold", "white, blue", "white", "light", "light", "~
$ eye_color  <chr> "blue", "yellow", "red", "yellow", "brown", "blue", "blue",~
$ birth_year <dbl> 19.0, 112.0, 33.0, 41.9, 19.0, 52.0, 47.0, NA, 24.0, 57.0, ~
$ sex        <chr> "male", "none", "none", "male", "female", "male", "female",~
$ gender     <chr> "masculine", "masculine", "masculine", "masculine", "femini~
$ homeworld  <chr> "Tatooine", "Tatooine", "Naboo", "Tatooine", "Alderaan", "T~
$ species    <chr> "Human", "Droid", "Droid", "Human", "Human", "Human", "Huma~
$ films      <list> <"A New Hope", "The Empire Strikes Back", "Return of the J~
$ vehicles   <list> <"Snowspeeder", "Imperial Speeder Bike">, <>, <>, <>, "Imp~
$ starships  <list> <"X-wing", "Imperial shuttle">, <>, <>, "TIE Advanced x1",~
```

This homework follows the four lectures of the week:

- **Q1**: writing functions (Day 20)
- **Q2**: `across()` over many columns (Day 21)
- **Q3**: `map()` / `walk()` over lists, files, and groups (Day 22)
- **Q4**: regular expressions (Day 23)

Question 1 – Writing functions

Recall the three types of functions discussed this week:

- A **summary function** returns a single value, so it lives inside `summarize()`.
- A **mutate function** returns a vector the same length as its input, so it lives inside `mutate()` / `filter()`.
- A **data frame function** takes a data frame as its first argument and *embraces* its column arguments with `{ }`.

Part (a): a summary function

Write a summary function `cv()` that returns the **coefficient of variation**, which is calculated by dividing the standard deviation by the mean. The coefficient of variation is a unitless measure of "how spread out relative to its size" a variable is. Give it an `na.rm` argument that defaults to `FALSE`, and pass that argument to **both** `sd()` and `mean()`.

Then **test** it on `c(2, 3, 4, 5, 5, 6, 7, 8)`. That vector has mean 5 and sd 2, so a correct `cv()` should return `0.4`.

```
# Write your R code here
```

Now use `cv()` to compare the spread of two columns in `starwars`: report the coefficient of variation of `height` and of `mass` in a single `summarize()` (remember `na.rm = TRUE`).

```
# Write your R code here
```

Which variable (column) is more variable relative to its average, height or mass? Is `cv()` a summary function or a mutate function, and how can you tell from its output?

[Write your answer here.]

Part (b): a data frame function

Write a **data frame function** `grouped_stats(df, group_var, num_var)` that groups `df` by `group_var` and returns the count `n`, plus the `mean`, `median`, and `sd` of `num_var`. **Embrace** both column arguments with `{ }`, use `na.rm = TRUE`, and end with `.groups = "drop"`.

Then call it on `starwars` to summarize `mass` by `sex`.

```
# Write your R code here
```

If you forgot the `{ }` and wrote `group_by(group_var)`, what error would you get? Why do you get this error?

[Write your answer here.]

Part (c): a plot function

Write a **plot function** `scatter(df, x, y)` that draws a scatterplot of `y` vs. `x` with a linear trend line. Embrace `x` and `y` inside `aes()`, use `geom_point()`, and add `geom_smooth(method = "lm", formula = y ~ x, se = FALSE)`.

For an automatic title, use `rlang::englue()`, which understands `{ }` and drops the *variable name* into a string:

```
label <- rlang::englue("{{ y }} vs. {{ x }}")
# ... then + labs(title = label)
```

Test it with `starwars |> scatter(height, mass)`.

```
# Write your R code here
```

> **Note**
>
> One character (a certain slug-like crime lord) has a mass over 1,300 kg and will stretch your y-axis. A `scale_y_log10()` transformation may be appropriate here.

Question 2 – `across()` over many columns

`across(.cols, .fns, .names)` repeats an action over a *set* of columns so you never copy-paste a line per column. Pass the function **name** with no `()`, and use an anonymous function `\(x) f(x, ...)` when you need to supply extra arguments.

With `summarize()`, compute the mean of every numeric column in starwars.

- Use `where(is.numeric)` to select the columns.
- Use an anonymous function to calculate the mean while setting `na.rm = TRUE`.
- Use the .names argument so the output columns append _mean to the original names (e.g., “{.col}_mean”).

```
# Write your R code here
# Hint: fill in the "..."'s
# starwars |>
#  summarize(
#    across(
```

```
#       .cols = ...,
#       .fns = \(x) f(x, ...),
#       .names = "{...}_..."
#     )
#   )
```

Part (b): two functions at once

For the columns `height`, `mass`, and `birth_year`, report both the mean and the number of missing values in one `summarize()`. Pass a **named list** to `.fns`, and use `.names = "{.fn}_{.col}"` so the output columns come out named `mean_height`, `n_miss_height`, `mean_mass`, and so on.

```
# Write your R code here
```

Which of those three columns has the most missing values? Why might that variable in particular be so often unknown?

[Write your answer here.]

Part (c): `across()` in mutate, `if_all()` in filter

Here is the `rescale01()` helper from class. Run this chunk so it's defined:

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
```

Use `across()` **inside** `mutate()` to rescale `height`, `mass`, and `birth_year` to run from 0 to 1, then `select()` those three columns to show the result.

```
# Write your R code here
```

Now use `if_all()` **inside** `filter()` to keep only the characters with **no missing value** in any of `height`, `mass`, `birth_year`, and `species`, and report how many rows remain with `nrow()`.

```
# Write your R code here
```

`across(a:c, f)` is shorthand for what longer, copy-pasted code? Give one reason the `across()` version is less error-prone.

[Write your answer here.]

Question 3 – `map()` / `walk()`

`map(x, f)` applies `f` to each element of `x` and returns a **list**. The typed variants (`map_dbl`, `map_int`, `map_chr`, `map_lgl`) return atomic **vectors** instead, and error loudly if the result isn't the type you promised.

Part (a): the typed map variants

Use the typed `map` variants to produce each of the following:

- the squares of `1:10`, as a **double** vector
- the length of each element of `list(1:5, 1:100, letters)`, as an **integer** vector
- `str_to_title()` of `c("ada", "grace")`, as a **character** vector

```
# Write your R code here
```

Part (b): map over a list-column

The `films` column of `starwars` is a **list-column**: each cell is a character vector of the films that character appeared in. `across()` can't easily reach inside those cells, but `map()` can.

Use `map_int()` to count how many films each character appeared in, add that as a new column `n_films` with `mutate()`, then show the **5 characters who appeared in the most films** (`name` and `n_films`, arranged descending).

```
# Write your R code here
```

Why is `map_int()` the natural tool here, and why would `across()` be awkward for this particular task?

[Write your answer here.]

Part (c): save one CSV per group with `walk2()`

You've been asked to export the archive as **one CSV file per `sex`**. The pattern is `group_nest()` to pack each group's rows into a list-column, `str_glue()` to build the file paths, and `walk2()` to write each `(data, path)` pair. We write into a temporary folder.

First, run this chunk to make a clean copy without the list-columns (which don't belong in a CSV) and set up the output folder:

```
sw <- starwars |>
  select(name, height, mass, sex, gender, species, homeworld)

out_dir <- tempdir()
```

Now `group_nest()` `sw` by `sex`, add a `path` column with `str_glue("{out_dir}/starwars-{sex}.csv")`, and print the resulting tibble.

```
# Write your R code here
```

Then use `walk2()` to write each group's data to its path, and confirm the files exist with `list.files()`:

```
# Write your R code here
# list.files(out_dir, pattern = "^starwars-.*\\.csv$")
```

Why do we use `walk2()` here instead of `map2()`? (Hint: do we care about the *return value*, or only about a *side effect*?)

[Write your answer here.]

Question 4 – Regular expressions

Regex does three jobs: **detect** whether a pattern occurs, **extract** the matching text, and **replace / clean** it away. We'll do all three, starting with the `name` column of `starwars`.

Part (a): detect

`str_subset(x, pattern)` returns the elements of `x` that match. Use it (and `str_detect()` where a count is asked for) to find:

- every character whose name contains `"Skywalker"`
- every character whose name **starts with** a vowel (anchor with `^` and a character class `[AEIOU]`)
- how many names contain a **digit** – `\\d` – using `sum(str_detect(...))`. (These are mostly the droids!)

```
# Write your R code here
```

Part (b): character classes, quantifiers, anchors

`starwars` names are tidy, so for raw pattern practice we'll use `words`, a vector of ~1,000 common English words that ships with `stringr`. Use `str_subset()` to find:

- words that **start with** `b`, `c`, or `d` (a character class at the start)
- words that **start with 3 consonants in a row** (`[^aeiou]{3}` anchored at the start)
- words with **3 or more vowels in a row** (`[aeiou]{3,}`)

```
# Write your R code here
```

Galactic comlink IDs happen to look exactly like Earth phone numbers, e.g. `(651) 696-6000` or `123-456-7890`. Complete the regex so that the first two strings match and the last two do **not**:

- `\\d` is a digit, `{3}` is exactly three of them
- `\\(?` and `\\)?` make literal parentheses **optional** (the `?` means "zero or one")
- `^` and `$` anchor the **whole** string

```
phones <- c("(651) 696-6000", "123-456-7890", "555.123.4567", "not a phone")

# phone_re <- "^\\(?\\d{3}\\)?[ -]?\\d{3}-\\d{4}$"
# str_detect(phones, phone_re) # should be TRUE TRUE FALSE FALSE
```

In your phone regex, what does the `?` after `\\(` and `\\)` accomplish? What would change if you removed the `^` and `$` anchors?

[Write your answer here.]

Part (c): extract and clean

Extract. Using the `fruit` vector from `stringr`, extract the **last vowel** of each fruit name. One clean way uses a lookahead, which is a vowel that is followed by only non-vowels until the end of the string. Fill in the character class:

```
# tibble(fruit = fruit) |>
#   mutate(last_vowel = str_extract(fruit, "[___](?=[^aeiou]*$)")) |>
#   head(6)
```

Clean. Here is a messy tibble of figures scraped from a galactic budget report. Turn it into real, usable data by droping the trailing `*` from `country` and turning `debt` and `percGDP` into numbers. `parse_number()` will ignore the `$`, commas, and the word `"billion"` for you; for the percent column, use `str_remove_all()` with a character class to strip the `,` and `%`.

```
debt <- tibble(
  country = c("World", "United States*", "Japan"),
  debt    = c("$56,308 billion", "$17,607 billion", "$9,872 billion"),
  percGDP = c("64%", "73.60%", "214.30%")
)
debt
```

```
# A tibble: 3 x 3
  country        debt            percGDP
  <chr>          <chr>           <chr>
1 World          $56,308 billion 64%
2 United States* $17,607 billion 73.60%
3 Japan          $9,872 billion  214.30%
```

```
# Write your R code here
```

Code appendix

```
library(tidyverse)
glimpse(starwars)
# Write your R code here
```

```r
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
# Hint: fill in the "..."'s
# starwars |>
#  summarize(
#    across(
#      .cols = ...,
#      .fns = \(x) f(x, ...),
#      .names = "{...}_..."
#    )
#  )
# Write your R code here
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
sw <- starwars |>
  select(name, height, mass, sex, gender, species, homeworld)

out_dir <- tempdir()
# Write your R code here
# Write your R code here
# list.files(out_dir, pattern = "^starwars-.*\\.csv$")
# Write your R code here
# Write your R code here
phones <- c("(651) 696-6000", "123-456-7890", "555.123.4567", "not a phone")

# phone_re <- "^\\(?\\d{3}\\)?[ -]?\\d{3}-\\d{4}$"
# str_detect(phones, phone_re) # should be TRUE TRUE FALSE FALSE
# tibble(fruit = fruit) |>
#   mutate(last_vowel = str_extract(fruit, "[___](?=[^aeiou]*$)")) |>
#   head(6)
debt <- tibble(
  country = c("World", "United States*", "Japan"),
  debt    = c("$56,308 billion", "$17,607 billion", "$9,872 billion"),
  percGDP = c("64%", "73.60%", "214.30%")
```

```
)
debt
# Write your R code here
```